

Tous les documents sont interdits. Tous les usages de dispositifs électroniques sont interdits, sauf évidemment ce qui est strictement requis en télé-examen, c'est-à-dire l'accès au sujet et l'envoi d'un scan de la copie papier. Par conséquent, tout usage de logiciel (éditeur de fichiers, compilateur, prouveur, etc) est interdit, même pour rédiger la copie d'examen. La copie d'examen doit être entièrement rédigée à la main.

Les règles de la logique de Hoare et la syntaxe de la logique du langage micro-C est rappelée dans l'annexe à la fin de ce document. Un barème est donné à titre indicatif.

Exercice 1 : Carré en logique de Hoare (9 points)

Question 1 : Exprimer par un triplet de Hoare que, si le programme suivant termine, alors il calcule dans c le carré de tout nombre n positif ou nul.

$c := 0; i := 0; \text{ while } i < n \text{ do } c := c + i; i := i + 1; c := c + i \text{ od}$

(1 point)

$\{n \geq 0\} c := 0; i := 0; \text{ while } i < n \text{ do } c := c + i; i := i + 1; c := c + i \text{ od } \{c = n^2\}$

Question 2 : En appliquant une par une les règles de la logique de Hoare, démontrer formellement dans un tableau que ce triplet de Hoare est valide.

(4,5 points)

Dans la logique de Hoare, une preuve de validité de ce triplet de Hoare est :

No	Triplet ou formule	Justification
1	$\{n \geq 0\} c := 0; i := 0; \text{ while } i < n \text{ do } c := c + i; i := i + 1; c := c + i \text{ od } \{c = n^2\}$	(seq.) 2 3
2	$\{n \geq 0\} c := 0 \{n \geq 0 \wedge c = 0\}$	(assign.)
3	$\{n \geq 0 \wedge c = 0\} i := 0; \text{ while } i < n \text{ do } c := c + i; i := i + 1; c := c + i \text{ od } \{c = n^2\}$	(seq.) 4 5
4	$\{n \geq 0 \wedge c = 0\} i := 0 \{n \geq 0 \wedge c = 0 \wedge i = 0\}$	(assign.)
5	$\{n \geq 0 \wedge c = 0 \wedge i = 0\} \text{ while } i < n \text{ do } c := c + i; i := i + 1; c := c + i \text{ od } \{c = n^2\}$	(conseq.) 6 7 8
6	$n \geq 0 \wedge c = 0 \wedge i = 0 \Rightarrow i \leq n \wedge c = i^2$	(voir (1))
7	$\{i \leq n \wedge c = i^2\} \text{ while } i < n \text{ do } c := c + i; i := i + 1; c := c + i \text{ od } \{i \geq n \wedge i \leq n \wedge c = i^2\}$	(while) 9
8	$i \geq n \wedge i \leq n \wedge c = i^2 \Rightarrow c = n^2$	évident
9	$\{i < n \wedge i \leq n \wedge c = i^2\} c := c + i; i := i + 1; c := c + i; \{i \leq n \wedge c = i^2\}$	(seq.) 10 11
10	$\{i < n \wedge c = i^2\} c := c + i \{i + 1 \leq n \wedge c + i + 1 = (i + 1)^2\}$	(voir (2))
11	$\{i + 1 \leq n \wedge c + i + 1 = (i + 1)^2\} i := i + 1; c := c + i \{i \leq n \wedge c = i^2\}$	(seq.) 12 13
12	$\{i + 1 \leq n \wedge c + i + 1 = (i + 1)^2\} i := i + 1 \{i \leq n \wedge c + i = i^2\}$	(assign.)
13	$\{i \leq n \wedge c + i = i^2\} c := c + i \{i \leq n \wedge c = i^2\}$	(assign.)

(1) L'implication

$$n \geq 0 \wedge c = 0 \wedge i = 0 \Rightarrow i \leq n \wedge c = i^2$$

est évidente : il suffit de remplacer i et c par 0 à droite de l'implication.

(2) Le triplet

$$\{i < n \wedge c = i^2\} c := c + i \{i + 1 \leq n \wedge c + i + 1 = (i + 1)^2\}$$

se justifie comme suit :

$$\overline{\{i + 1 \leq n \wedge c + i + i + 1 = (i + 1)^2\} c := c + i \{i + 1 \leq n \wedge c + i + 1 = (i + 1)^2\}}$$

selon la règle (*assignment*). Or $i + 1 \leq n$ est équivalent à $i < n$ et $c + i + i + 1 = (i + 1)^2$ se simplifie successivement en $c + 2i + 1 = i^2 + 2i + 1$ puis $c = i^2$.

Question 3 : Définir une fonction C

```
int square(int n)
```

qui implémente le programme de ce triplet de Hoare et retourne la valeur qu'il calcule dans sa variable c .

(1 point) Voir le code C du listing 1.

Question 4 : Transformer la précondition et la postcondition de ce triplet de Hoare en annotations micro-C pour cette fonction.

(1 point) Voir le listing 1.

Question 5 : Proposer un invariant micro-C assez précis pour prouver cette postcondition.

(1 point) Voir le listing 1.

Question 6 : Proposer un variant micro-C permettant de prouver la terminaison de cette fonction C.

(0,5 point) Voir le listing 1.

```
int square (int n) {
    /*@ requires n >= 0;
       ensures result == n*n; */
    int c = 0;
    int i = 0;
    while (i < n) {
        /*@ invariant c == i*i && i <= n;
           variant n-i; */
        c = c+i;
        i = i+1;
        c = c+i;
    }
    /* Here i == n and c == i*i, so c == n*n */
    return(c);
}
```

Listing 1 – Fichier `square.c`

Exercice 2 : Formules sur les entiers (5 points)

Le but de cet exercice est de formaliser par des lemmes micro-C des propriétés de parité des entiers. Certains lemmes peuvent aider à en démontrer d'autres.

Question 1 : Définir en syntaxe micro-C deux prédicats `even` et `odd` indiquant respectivement si un entier est pair ou impair, en utilisant seulement la logique du premier ordre, l'addition, la multiplication et l'égalité.

(1 point)

```
predicate even (int n) = exists k. n == 2 * k;
predicate odd (int n) = exists k. n == 2 * k + 1;
```

Question 2 : Formaliser chacune des propriétés suivantes par un lemme dans la logique de micro-C.

1. Tout entier s'écrivant comme le double d'un autre entier est pair.

(0,5 point)

```
goal even_2k: forall k. even (2 * k);
```

prouvé automatiquement.

2. Tout entier de la forme $2k + 2$ est pair.

(0,5 point)

```
lemma fact2: forall k. 2*k+2 == 2*(k+1);
goal even_2kp2easier: forall k. even (2*k+2);
```

(pas demandé) Le second lemme seul n'est pas prouvé automatiquement par Alt-Ergo en 5 secondes, mais il est prouvé automatiquement grâce au lemme ajouté (pas demandé), qui aide le prouveur à factoriser.

3. Tout entier pair n'est pas impair.

(0,5 point)

```
lemma even_not_odd: forall n. even(n) -> !odd(n);
```

prouvé automatiquement.

4. Le successeur de tout entier pair est impair.

(0,5 point)

```
lemma even_succ_odd: forall n. even(n) -> odd(n+1);
```

prouvé automatiquement.

Question 3 : En n'utilisant que la logique du premier ordre, la multiplication et l'égalité, définir en micro-C un prédicat `divides` indiquant qu'un entier d divise un entier n .

(1 point, que 0,5 point s'il manque `exists`)

```
predicate divides (int d, int n) = exists q. n == d * q;
```

Question 4 : Formaliser la propriété suivante par un lemme dans la logique de micro-C : *Un entier est pair si et seulement s'il est divisible par 2.*

(1 point)

```
lemma even_divides_eq: forall n. even(n) <-> divides(2,n);
```

prouvé automatiquement.

Exercice 3 : Tableau aléatoire décroissant (6 points)

On rappelle que l'expression `rand()` fournit un entier pseudo-aléatoire positif ou nul. Le programme C suivant remplit tout tableau d'entiers `t` avec des nombres aléatoires **strictement décroissants**.

```
void random_decreasing_array(int t[], int n) {
    int i = 0;
    int v = 0;
    while (i < n) {
        v = v - rand() - 1;
        t[i] = v;
        i = i + 1;
    }
}
```

Question 1 : Spécifier par une précondition micro-C que le paramètre `n` de cette fonction est la longueur du tableau `t`.

(1 point) `requires n == length(t);`

Question 2 : Pour cette fonction, proposer une postcondition micro-C qui décrit que le tableau résultant est strictement décroissant.

(1 point) `ensures forall i. 0 < i < n -> t[i-1] > t[i]`

Question 3 : Dans un invariant de boucle en syntaxe micro-C, spécifier les bornes les plus précises possible pour la variable `i`.

(1 point) `invariant 0 <= i <= n;`

Question 4 : Pour `i` non nul, spécifier l'invariant de boucle le plus précis possible pour la valeur de la variable `v`.

(1 point, 0 point si cet invariant n'est pas une égalité ou s'il manque `i > 0 ->`)
`invariant 0 < i -> v == t[i-1];`

Question 5 : Spécifier un invariant de boucle assez précis pour permettre de démontrer automatiquement la postcondition de la question 2 (avec l'aide des invariants précédents).

(1 point) `invariant forall j. 0 < j < i -> t[j-1] > t[j];`

Question 6 : Spécifier un variant de boucle permettant de démontrer la terminaison de cette fonction.

(1 point) `variant n - i;`